

Table of Contents

Studium — Agent Runtime and Orchestration Specification	1
1. Overview	2
2. Scope and non-goals.....	2
3. Design principles	3
4. Technology stack	4
5. Runtime topology	5
6. The agent contract	6
7. Session state machine	8
8. The Orchestrator.....	12
9. The Curator	14
10. The Lecturer.....	16
11. The Tutor.....	18
12. The Evaluator	21
13. The Confusion-Tracker	23
14. The Reviewer	25
15. The tutorial primitives.....	26
16. Session lifecycle	28
17. Prompt caching strategy	29
18. Model routing	30
19. Cost accounting.....	31
20. Concurrency, streaming, and interruption	32
21. Error handling and degradation	33
22. Observability.....	35
23. Testing strategy	35
24. Version history.....	36
25. Forward references and open questions.....	37
End of specification	38

Studium — Agent Runtime and Orchestration Specification

Subsystem 2 of 7. Version 1.0. Status: build-ready.

Written against the data layer specification v1.1. Every schema reference in this document points to §6 of that specification. Any need to change a schema surfaced during agent implementation is recorded as a v1.2 candidate in §22 rather than being applied silently.

1. Overview

This document specifies the agent runtime for Studium: the set of coordinated LLM-backed services that produce every learner-facing behaviour in the system. Where subsystem 1 specified what is persisted, this specifies what runs. Six subsystem specs follow this one, but four of them — retrieval, frontend, content ingestion, evaluation — depend materially on decisions locked here, so this is the load-bearing spec after the data layer.

A senior engineer with the data layer v1.1 spec and this document should be able to build a working agent runtime that reads and writes the tables in §6 of v1.1, streams to a frontend that has not yet been built, and produces the four MVP interaction modes (lecture, tutorial, lab, confusion journal) end-to-end. The frontend spec that follows will describe how the client consumes the streaming interface defined here; the retrieval spec will describe the interior of the `retrieve_passages` tool this document treats as a black box.

The agent runtime is the part of the system most likely to change under iteration once real learners are using it. Prompt wording will be tuned, model choices will be re-evaluated, primitive semantics will be sharpened. What this spec locks is the **structure** that permits those changes without downstream breakage: the agent boundaries, the interfaces they expose, the state machine that coordinates them, the observability that lets tuning be honest. Prompt text is illustrative and versioned; agent contracts are the contract.

2. Scope and non-goals

In scope.

- The seven agents that make up the group: Orchestrator, Curator, Lecturer, Tutor, Evaluator, Confusion-Tracker, Reviewer. Full profiles, prompt structures, tool contracts, and coordination protocol.
- The session state machine: lecture, tutorial, lab, review, office hours, summative assessment, orientation. States, transitions, guards, and events.
- The tutorial interaction primitives: `explain_differently`, `prove_it_to_me`, `where_does_this_fit`, `vocabulary_check`, `show_worked_example`, `let_me_try_one`, `why_does_this_matter`, `im_lost`. What each does at the agent-level and how they route through the Orchestrator.
- Streaming and interruption model. How mid-lecture “raise your hand” interrupts terminate one agent’s stream and hand control to another without dropping session context.
- Prompt caching strategy. The byte-stable prefix design that keeps per-turn cost low.
- Model routing. Per-agent choices with pluggable providers.
- Cost accounting. Every LLM call, every source, every attribution rule.

- Error handling, retries, and degradation. What happens when a model call fails, when rate-limits hit, when content filters trip, when the client disconnects mid-stream.
- Observability. Langfuse trace structure, per-agent metrics, cost dashboards.

Explicitly out of scope.

- The interior of the `retrieve_passages` tool. Chunking, embedding, hybrid search, reranking, citation resolution live in the Retrieval spec (subsystem 3). This document treats retrieval as a black box with a defined contract.
- The frontend’s rendering of agent output. How SSE streams are consumed, how the tutorial primitive UI is presented, how the “raise your hand” gesture is detected on the client side — all live in the Frontend spec (subsystem 4).
- Content ingestion. How source PDFs become chunks and how the concept graph is authored live in Content Ingestion (subsystem 5). The agents read what ingestion produces; they do not produce it themselves.
- Golden datasets and per-agent regression evaluations. Named here as consumers of what Evaluation (subsystem 6) will define, but not specified in this document.
- Deployment topology. Where the FastAPI process runs, how it scales, secrets management — all in Infrastructure (subsystem 7).
- Voice. Text-first for MVP. The agent contracts are text-in, text-out; voice arrives as a subsequent layer that transcribes to text before entering this system and synthesizes from text before leaving it. Voice-specific state (barge-in latency, ASR partial handling) is out of scope until the voice spec exists.

3. Design principles

Every design choice below is downstream of one of these. Any subsequent spec that appears to violate one is either wrong or has surfaced a problem this document should address; there is no third case.

Single agent per turn. Exactly one agent produces the reply to a given learner turn. Multiple agents may participate in preparing that reply (the Curator may hand context to the Lecturer, the Confusion-Tracker may run asynchronously after) but only one agent’s output is streamed to the learner as the response. This makes cost attribution, prompt caching, and error handling tractable; the alternative — multiple agents composing an output — creates a state-machine problem that is not worth the added quality until the runtime is much more mature than MVP.

No cross-agent prompt context. Each agent’s prompt is assembled from the data layer (`session_turns`, `mastery_events`, `journal_entries`) and never from another agent’s system prompt or internal reasoning. The Tutor does not see the Curator’s plan-generation prompt; the Evaluator does not see the Tutor’s encouraging framing. This is the “agent context isolation” principle from data layer §3, enforced at the runtime level. If an agent needs information another agent produced, that information passes through the schema — as a `session_turns` row, a `journal_entries` update, or an explicit context field — never as prompt-to-prompt sharing.

Every LLM call is accounted for at the point of the call. The moment an agent invokes a model, it writes an `agent_traces` row with tokens, cost, latency, and cache-hit statistics — in the same transaction that writes the corresponding `session_turns` row where applicable. There is no “we’ll reconcile costs later” pathway. This is what makes the widened cost ledger (data layer §6.12) meaningful.

Structured output over free-form. Where a downstream consumer (the schema, another agent, the frontend) needs to make a decision on part of an agent’s output, that part is emitted via a structured output tool with a validated schema, not extracted from prose after the fact. The existing draft already does this well with Pydantic-typed `messages.parse`; the runtime formalizes it as a pattern.

Prompt caching is architectural, not opportunistic. Every agent’s system prompt is split into a byte-stable cached prefix (persona, task instructions, concept grounding for the current focus) and a per-turn suffix (recent session context, current learner input). The prefix is designed to hit cache on every call after the first within a session. The existing draft’s `cached_system()` helper is the model; the runtime extends it per-agent.

Interruption is a first-class control flow event, not an error. When a learner interrupts a streaming lecture, the runtime does not treat that as a stream failure. It transitions cleanly through a defined state (`LECTURING` → `INTERRUPTED` → `PAUSED_FOR_QUESTION`), finishes the current sentence, preserves the interruption point for later resumption, and hands off to the Tutor. This is spec’d because getting it wrong looks broken to the learner in a way that is uniquely damaging to trust.

Degradation is designed, not accidental. When a model call fails after retries, when a rate limit is hit, when a budget cap trips, the learner sees a coherent response — a spoken apology, a request to try again, a suggestion to review while the system recovers — never a raw error. Each failure mode has a specified degradation path in §16.

The Orchestrator is a coordinator, not a reasoner. The Orchestrator holds session state and routes turns to agents. It does not itself call a language model to decide who should speak next (with two exceptions noted in §5). Routing decisions are deterministic functions of the session state machine and the learner’s explicit signal (primitive button, message intent classified by a small Haiku call). This keeps the coordination layer cheap, fast, and debuggable.

Cost caps are enforced at the runtime boundary. Before any billable operation, the runtime consults the widened cost check from data layer §8 and refuses to proceed if the user’s hard cap is exceeded. The refusal is graceful — the learner is told, given options, and their session is preserved. This is the difference between a project ended by a \$2,000 surprise and a project bounded by a \$100 hard cap.

4. Technology stack

Runtime. Python 3.11+ inside the FastAPI process. Agents are Python classes with async methods, not separate services. Rationale for in-process: MVP has three users, coordination latency between separate services would dominate the per-turn cost budget, and the

observability story is far simpler with a single process. When Studium reaches a scale where in-process contention matters (rough trigger: 50+ concurrent sessions), agents can be extracted to separate services behind the same interfaces defined here.

Model provider. Anthropic SDK, anthropic Python package, 0.40 or later. Route through the existing `client.py` helper from the draft, extended to support agent-specific routing (§18).

Model choices (locked at v1.0 of this spec; per-agent overrides in §18): - Claude Opus 4.8: Lecturer, Tutor, Curator, Evaluator (grading), Reviewer (retrieval-prompt generation) - Claude Haiku 4.5: Confusion-Tracker, Orchestrator's intent classifier, Evaluator (practice-question checks) - Embeddings: Voyage-3 via Voyage AI SDK (used by Retrieval, not directly by agents)

Structured output. Anthropic's tool-use with `output_format` (Pydantic model class), matching the existing draft's `messages.parse` pattern. Every non-streaming agent method returns a validated Pydantic instance. Streaming methods emit text with structured metadata as tool calls where needed.

Observability. Langfuse for LLM-specific tracing (per-agent, per-turn, per-session views). OpenTelemetry for HTTP request tracing (FastAPI middleware). Both feed the same trace ID so a support conversation can navigate from a Langfuse trace to the containing HTTP request and back.

Streaming. Server-Sent Events over HTTP from FastAPI to the client. WebSockets not used — SSE is sufficient for the one-directional streaming pattern (server-to-client only during agent output; client-to-server is separate HTTP requests). WebSocket support may be reconsidered when voice ships.

Async model. All agent methods are `async def`. The FastAPI handler for a learner turn opens a request context, runs the Orchestrator's `handle_turn()` coroutine, and returns either a streaming response (for Lecturer/Tutor turns) or a JSON response (for background operations like session close).

5. Runtime topology

The agent runtime lives in `backend/studium/agents/`. Each agent is a Python module implementing a standard interface. Coordination lives in `backend/studium/orchestration/`.

Directory layout.

```
backend/studium/
├── agents/
│   ├── base.py           # Agent ABC and shared utilities
│   ├── orchestrator.py   # Session state machine, routing
│   ├── curator.py        # Syllabus, next-topic, session opening
│   ├── lecturer.py       # Structured exposition, comprehension checks
│   └── tutor.py          # Socratic dialogue, primitive handling
```

—	evaluator.py	# Grading (practice, assessment)
—	confusion_tracker.py	# Async gap monitoring
—	reviewer.py	# Spaced-repetition retrieval prompts
—	orchestration/	
—	state_machine.py	# Session states and transitions
—	streaming.py	# SSE emitter, sentence-boundary interruption
—	primitives.py	# Interaction primitive dispatch
—	handoff.py	# Agent-to-agent context assembly
—	budget.py	# Cost cap enforcement
—	llm/	
—	client.py	# Anthropic client, per-agent routing
—	prompts.py	# Cached prefix assembly
—	retries.py	# Retry policy, backoff
—	traces.py	# agent_traces writer
—	session/	
—	lifecycle.py	# Session open, close, summary
—	memory.py	# Cross-session context assembly
—	budget_gate.py	# Pre-flight budget check

Two exceptions to the “Orchestrator does not reason” rule. The Orchestrator does invoke Haiku for two narrow classification tasks: (a) intent classification when the learner sends free-form text mid-lecture — is this a question, a comment, a request to skip, an interruption? — and (b) sentence-boundary detection when interrupting a stream, if the deterministic tokenizer-based boundary detector fails (rare, but happens on unusual punctuation patterns from LaTeX or code). Both calls are Haiku, both are bounded to a single turn, both write agent_traces rows attributed to the orchestrator agent identity (from the agent_identity enum in data layer §6.0).

6. The agent contract

Every agent implements the same interface. This is not merely tidiness — the uniform contract is what lets the Orchestrator route without special-casing per agent, what lets Langfuse trace uniformly, and what lets a future agent be added by writing one class rather than editing every caller.

Agent abstract base class (backend/studium/agents/base.py):

```
from abc import ABC, abstractmethod
from typing import AsyncIterator, Optional
from pydantic import BaseModel
from studium.session.context import SessionContext
from studium.llm.traces import TraceRecord

class AgentInput(BaseModel):
    """Standard input to any agent method call.

    session_context is the full read-side context: session row, focus
    concept,
    recent turns, mastery snapshot, journal entries. Agents receive this
```

```

    already-assembled; they never query the database themselves.
    """
    session_context: SessionContext
    kind: str # method dispatch: 'deliver_segment',
'answer', 'grade', ...
    payload: dict # method-specific arguments, per-agent
schema

class AgentOutput(BaseModel):
    """Standard output for non-streaming calls."""
    text: str # what would be shown to the learner
    structured: Optional[BaseModel] = None # method-specific structured
data
    trace: TraceRecord # cost, tokens, latency, model, cache
stats
    tool_effects: list["ToolEffect"] = [] # journal writes, mastery
updates, ...

class ToolEffect(BaseModel):
    """A side-effect the agent has decided should happen. Applied by the
    Orchestrator after the call completes, so retries do not double-apply.
    """
    kind: str # 'update_journal',
'record_mastery_evidence', ...
    payload: dict

class Agent(ABC):
    identity: str # matches agent_identity enum value
    model: str # default model; per-call override
possible

    @abstractmethod
    async def handle(self, input: AgentInput) -> AgentOutput:
        """Non-streaming call. Returns fully-formed output."""

    async def handle_streaming(
        self, input: AgentInput
    ) -> AsyncIterator["StreamChunk"]:
        """Streaming call. Default implementation raises NotImplementedError;
        agents that stream (Lecturer, Tutor) override.
        """
        raise NotImplementedError(f"{self.identity} does not support
streaming")

class StreamChunk(BaseModel):
    kind: str # 'text', 'tool_effect', 'trace', 'end'
    payload: dict

```

The SessionContext shape. Assembled once at the start of every turn by orchestration/handoff.py, passed to whichever agent runs. This is the object that carries everything an agent needs to know from the data layer without giving it direct database access.

```
class SessionContext(BaseModel):
    session: LearningSessionRow
    learner: UserRow
    learner_subject: LearnerSubjectRow
    subject: SubjectRow

    focus_concept: Optional[ConceptRow]
    focus_neighborhood: list[ConceptRow] # prerequisites + immediate dependencies

    recent_turns: list[SessionTurnRow] # last 12 turns of this session
    mastery_snapshot: dict[UUID, float] # concept_id -> p_known_decayed
    open_journal_entries: list[JournalEntryRow]

    prior_session_summary: Optional[SessionSummaryRow]
    retrieval_check_result: Optional[RetrievalCheckRow]

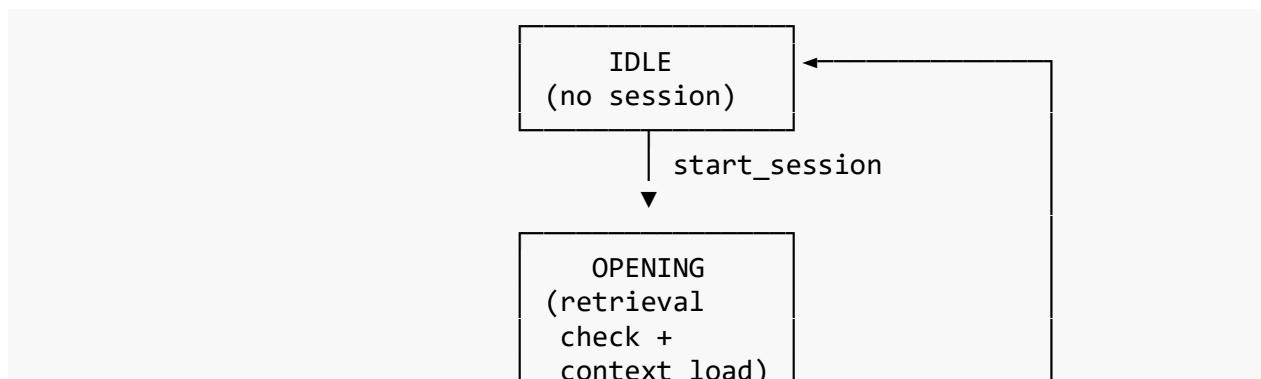
    # Cache key components for the agent's system prefix
    grounding_version: str # subject.version + focus_concept.updated_at hash
```

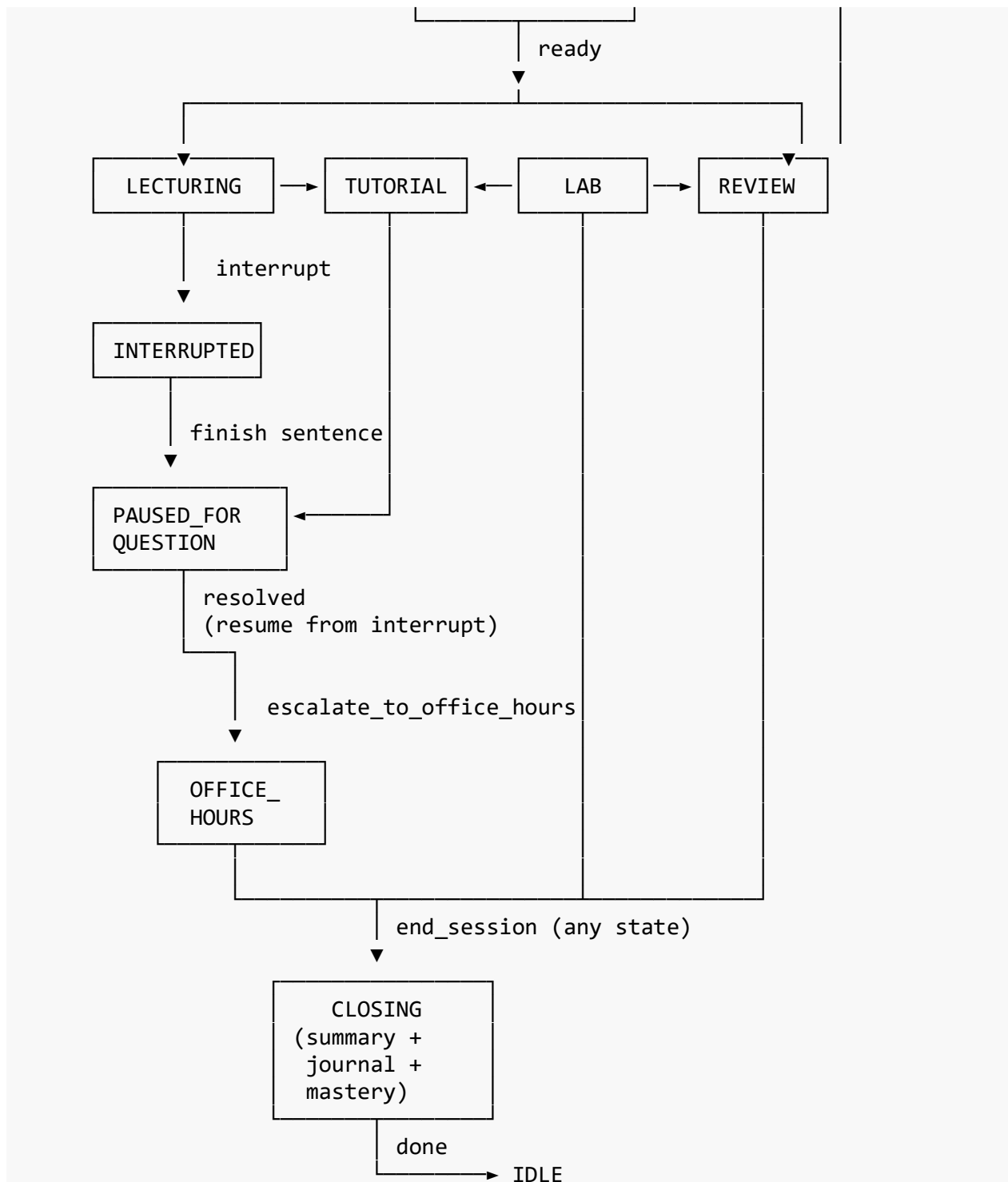
The context is deliberately generous. It is cheaper to assemble a wider context than an agent uses than to have an agent discover mid-turn that it needs a field the context does not carry.

Method dispatch. Each agent defines a set of kind values it accepts in `AgentInput.kind`. Unknown kinds raise `AgentDispatchError`, caught by the Orchestrator as a programmer error (never a learner-visible one). The kinds per agent are enumerated in each agent's section below.

7. Session state machine

The Orchestrator maintains one state machine per active session. States and transitions:





State registry (mirrors session_mode enum from data layer §6.0 where applicable, with runtime-only states added):

State	Persisted session_mode	Description
-------	------------------------	-------------

State	Persisted session_mode	Description
IDLE	—	No active session for the learner
OPENING	(mode being started)	Assembling context, running retrieval check
LECTURING	lecture	Lecturer is delivering a segment
TUTORIAL	tutorial	Tutor is running a Socratic exchange (proactive, not interruption)
LAB	lab	Learner is producing work; Evaluator watching
REVIEW	review	Reviewer is running due cards
INTERRUPTED	(unchanged from prior)	Lecture stream is finishing current sentence before pausing
PAUSED_FOR_QUESTION	(unchanged from prior)	Tutor is answering an interruption; will resume prior state
OFFICE_HOURS	office_hours	Escalated from PAUSED_FOR_QUESTION; standalone Tutor session
SUMMATIVE_ASSESSMENT	summative_assessment	Evaluator running proctored assessment
CLOSING	(mode being closed)	Generating summary, writing journal, updating mastery

Transitions. Table below. *guard* is a boolean condition; *effect* is what runs during the transition. All transitions write a state-transition row to a runtime-only in-memory log (not persisted; the *session_turns* sequence carries the persistent record).

From	Event	Guard	Effect	To
IDLE	start_session(mode, focus)	Budget cap not exceeded	Create learning_sessions row; assemble SessionContext	OPENING
OPENING	context_ready	Prior summary exists	Curator generates retrieval check via Reviewer	(retrieval-check subroutine)
OPENING	context_ready	No prior summary	—	mode's state
LECTURING	segment_complete	More segments queued	Next segment via Lecturer	LECTURING
LECTURING	segment_complete	Segments exhausted	—	TUTORIAL or LAB per Curator

From	Event	Guard	Effect	To
LECTURING	learner_interrupt	Stream in progress	Signal stream to finish sentence	INTERRUPTED
INTERRUPTED	sentence_boundary_reached	—	Cancel remaining stream tokens; record session_turns for the delivered portion	PAUSED_FOR_QUESTION
PAUSED_FOR_QUESTION	question_resolved	Learner signals satisfaction	Restore SessionContext snapshot; resume Lecturer from interruption point with brief recap	LECTURING
PAUSED_FOR_QUESTION	escalate	Learner says “this needs more time”	—	OFFICE_HOURS
TUTORIAL	primitive_invoked	Primitive is let_me_try_one	—	LAB
LAB	answer_submitted	Evaluator: correct	Record mastery_events (practice_correct)	LAB or TUTORIAL per Curator
LAB	answer_submitted	Evaluator: incorrect after N attempts	Confusion-Tracker writes journal_entries; Tutor takes over	TUTORIAL
any	end_session	Not CLOSING	—	CLOSING
CLOSING	close_complete	—	Write session_summaries, update learning_sessions.ended_at, run per-user cost roll-up delta	IDLE

Timeouts. If a state persists longer than the session's `target_duration_minutes + 15` without an active turn, the Orchestrator emits `end_session` with `end_reason = 'idle_timeout'`. This prevents zombie sessions from holding budget or context.

Persistence. The state machine's current state is not stored in a dedicated column. It is a runtime attribute of the in-memory Orchestrator instance for the session, reconstructible from the `session_turns` tail if the process restarts. This is a deliberate choice: adding a `learning_sessions.current_state` column would create a synchronization bug surface (state in memory disagrees with column) and is not needed at MVP scale where the process is single-node. When Studium goes multi-node, the state either lives in Redis or is reconstructed from turns on request pickup.

8. The Orchestrator

The Orchestrator is the coordinator. It owns the session state machine, assembles `SessionContext` for each turn, routes to agents, applies their `ToolEffects`, and streams to the client.

Interface (`agents/orchestrator.py`):

```
class Orchestrator(Agent):
    identity = "orchestrator"
    model = "claude-haiku-4-5"  # for the intent classifier

    async def handle_turn(
        self, session_id: UUID, learner_input: LearnerInput
    ) -> AsyncIterator[StreamChunk]:
        """Main entry point. Called by the FastAPI turn handler.
        Yields StreamChunks to the SSE emitter.
        """

    async def start_session(
        self, user_id: UUID, mode: SessionMode, focus_concept_id:
Optional[UUID]
    ) -> UUID:
        """Create a session, run OPENING, return session_id."""

    async def end_session(
        self, session_id: UUID, reason: str
    ) -> None:
        """Run CLOSING, persist summary, release resources."""
```

Turn handling in detail. The `handle_turn` coroutine:

1. Loads or reconstructs the session's state and `SessionContext`.
2. Runs the budget gate (`session/budget_gate.py`); if the daily hard cap would be exceeded by an expected worst-case turn cost, yields a degradation chunk and returns.

3. Classifies the learner's intent if the input is free-form text (`_classify_intent()`; one Haiku call). Intents: question, answer, comment, interrupt, next, back, primitive:<name>, end_session.
4. Applies the state-machine transition matching the (state, intent) pair.
5. Dispatches to the appropriate agent's `handle` or `handle_streaming` method with the `SessionContext`.
6. Streams the agent's output to the client via the SSE emitter, watching for interrupt signals.
7. On completion, applies the agent's `ToolEffects` to the database in a single transaction.
8. Writes the `session_turns` row for the turn.
9. Runs the Confusion-Tracker asynchronously (fire-and-forget) if the turn's content warrants it.

Intent classifier prompt. Bounded by a Pydantic output schema. Full prompt template:

System (cached; byte-stable across all classifications):

You classify a single learner utterance during a Studium session. Return exactly one intent from the enumerated set.

INTENT DEFINITIONS

- question: the learner is asking for information or clarification.
- answer: the learner is responding to a question the system asked.
- comment: the learner is making an observation without expecting a reply that changes the session's direction.
- interrupt: the learner wants the current agent to stop what it is doing (usually mid-lecture).
- next: the learner wants to proceed to the next segment or topic.
- back: the learner wants to return to a previous segment or topic.
- primitive:<name>: the learner is invoking a named tutorial primitive.
Valid names: `explain_differently`, `prove_it_to_me`, `where_does_this_fit`, `vocabulary_check`, `show_worked_example`, `let_me_try_one`, `why_does_this_matter`, `im_lost`.
- end_session: the learner wants to stop for the day.

If the utterance is ambiguous, prefer 'question' when it contains a question mark or interrogative phrasing, 'comment' otherwise.

User (per turn):

Session state: {state}

Last system message (truncated 200 chars): {last_system}

Learner utterance: {utterance}

Output schema (tool call, required):

```
IntentClassification {
  intent: enum
```

```

    confidence: float in [0, 1]
    reasoning: string (one sentence)
}

```

Confidence below 0.7 falls back to a rule-based classifier that examines punctuation and keyword patterns. This gives a deterministic escape hatch when the model wavers.

ToolEffect dispatch. After an agent returns, the Orchestrator applies each effect in the order returned. Effects and their handlers:

ToolEffect kind	Handler	Data-layer target
record_mastery_evidence	mastery.apply_evidence()	mastery_events + concept_mastery update
update_journal	journal.upsert_entry()	journal_entries + journal_events
record_content_artifact	content.persist_artifact()	content_artifacts + content_citations
schedule_review	review.schedule_card()	review_cards update
record_portfolio_item	portfolio.append_item()	portfolio_items
flag_for_review	review_queue.enqueue()	content_review_queue

All effects for a turn are applied in one transaction. If any effect fails, all effects for the turn are rolled back and an error is logged; the turn itself (the session_turns row) still writes, so the trace remains complete.

9. The Curator

The Curator owns curriculum sequencing for a subject. It answers three questions: what is next, what is the plan, and how should this session open.

Kinds accepted.

Kind	Purpose	Streaming
open_session	Assemble the session's opening: retrieval check + agenda	No
next_topic	Given current mastery, pick the next concept to work on	No
revise_syllabus	Regenerate the learner's syllabus after significant mastery change	No
select_stance	Choose a pedagogical stance for a Lecturer call	No

System prompt structure (cached prefix + per-call suffix):

```
[CACHED PREFIX – byte-stable per (subject_id, subject_version, learner_id)]
```

You are the Curator for the subject "{subject.title}", version {subject.version}.

You own the ordering, pacing, and shape of one learner's path through this subject. You do not teach; you decide what teaching happens next.

SUBJECT SHAPE

{subject.long_description}

CONCEPT GRAPH (topological summary)

{numbered list of concepts with depth, load-bearing status, prerequisites}

LOAD-BEARING CONCEPTS (prioritize mastery here)

{list of concepts where is_load_bearing = true}

THIS LEARNER

Preferred pace: {learner.preferences.pace}

Stated goals: {learner.profile.stated_goals}

Preferred stance (if any): {learner.subject_prefs.preferred_stance}

DECISION PRINCIPLES

- Prefer concepts whose prerequisites are all above 0.85 mastery (the unlock threshold from the data layer's mastery model).
- Prefer concepts with high load-bearing weight before their dependents.
- If mastery has been stagnant on a concept for more than 2 sessions, pivot to a different stance or an adjacent concept rather than repeating.
- Session length is a hard budget. Aim for 1-3 concepts per 90 minutes at standard depth.
- The learner's stated goals are a soft steer, not a hard constraint. If the goal is unreachable without a prerequisite the learner has not mastered, say so and route to the prerequisite.

[PER-CALL SUFFIX]

Current session mode: {mode}

Session duration remaining: {minutes_remaining} minutes

CURRENT MASTERY SNAPSHOT (concept -> p_known_decayed)

{mastery_snapshot as list, sorted by decayed mastery ascending}

RECENT CONFUSION (open journal entries)

{recent open journal entries: concept, summary, first_seen_at}

RETRIEVAL CHECK RESULT (if opening a session with prior context)

{overall_score, weak_prompts}

Your task: {task-specific instruction}

Structured output varies by kind. For open_session:

```
class SessionOpening(BaseModel):
```

```
    focus_concept_id: UUID
```

```
    session_agenda: list[str]
```

```
    prior_session_note: Optional[str]
```

```
    # 3-6 items, Learner-visible
```

```
    # what to say about last time, if
```

```

applicable
    retrieval_check_prompts: list[RetrievalPrompt] # empty if no prior
context
    initial_mode: Literal['lecture', 'tutorial', 'review', 'lab']
    justification: str # internal, for trace

```

For next_topic:

```

class NextTopic(BaseModel):
    concept_id: UUID
    mode: Literal['lecture', 'tutorial', 'lab']
    stance: Literal['formal', 'intuitive', 'applied', 'historical',
'default']
    reason: str # one sentence, internal

```

Cost profile. Curator calls happen at session open (~1) and at topic transitions (~2-4 per session). Each call is ~5-15k input tokens (mostly cached), ~500-1500 output tokens. Cached prefix is large — the concept graph summary is expensive to produce once but effectively free thereafter within a session.

Failure modes. If the Curator produces a next_topic that fails the unlock check (deterministic post-processing catches this), the Orchestrator asks the Curator once more with an explicit “your previous choice was locked, prerequisites are: [...]” appendix. If the second attempt also fails, fall back to the deterministic mastery.suggest_next_unlocked(learner_subject_id) from application code, which picks the lowest-depth unlocked concept with the least mastery. The Curator’s failure is logged; two failures in a row on the same session flags the trace for reviewer attention.

10. The Lecturer

The Lecturer delivers structured exposition. It works from a concept node, pulls source material via retrieve_passages, and produces a lecture segment grounded in the corpus.

Kinds accepted.

Kind	Purpose	Streaming
deliver_segment	Produce and stream one lecture segment for the current concept	Yes
generate_check	Produce a comprehension check for the current segment	No
re_explain	Regenerate an explanation in a different stance (from explain_differently)	Yes
worked_example	Produce a fully worked example (from show_worked_example)	Yes

System prompt structure.

```

[CACHED PREFIX – byte-stable per (concept_id, stance, grounding_version)]

You are the Lecturer for Studium. You deliver structured exposition of one

```

concept at a time to a serious adult learner. You are teaching, not chatting.

CONCEPT

{concept.title}

CONCEPT CONTEXT

{concept.long_description}

Prerequisite concepts the learner has mastered:

{prerequisites with mastery > 0.85}

Related concepts the learner has seen (do not re-teach; may reference):

{concepts touched in prior sessions}

CANONICAL SOURCES (cite from these; do not import outside content)

{retrieved passages, numbered, with source titles and page refs}

STANCE

{stance-specific instructions, e.g. for 'formal':

"Prefer precise definitions and derivations. Use notation from the sources.
Motivate before formalizing. If a proof is needed, sketch structure before detail."

}

TEACHING PRINCIPLES

- Start where the learner is. If a prerequisite is at exactly 0.85, remind briefly; if it is at 0.99, do not.
- One idea per segment. If the segment would exceed 350 words, split.
- Every non-trivial claim is grounded in the sources listed above. Cite by passage number: [P3], [P7-P8].
- Worked examples are complete: state the setup, do each step, name what each step accomplishes.
- End every segment with a brief anchor: what was learned, what comes next.
- Do not tell the learner what they already know. Do not restate the concept title as a summary.

CONSTRAINTS

- 200-400 words per segment, hard limit.
- Markdown, minimal formatting. Use inline code fences for symbols, math via LaTeX in $\dots$ or $\$ \$ \dots \$ \$$, tables only when comparing three or more items.
- Never invent citations. If a claim needs a source you do not have, either omit the claim or note that this is a broader-context remark.

[PER-CALL SUFFIX]

Segment index: {n} of {total} in the current sequence

Previous segment ended with: {last_segment_anchor}

Learner interruption (if resuming): {interruption_context or 'none'}

Task: {task-specific instruction}

Grounding retrieval. Before producing a segment, the Lecturer calls `retrieve_passages(concept_id, stance, k=6)` (interior defined in the Retrieval spec). The retrieved passages become the numbered source list in the system prompt. If retrieval returns fewer than three passages, the Lecturer notes to the trace that grounding is thin and the segment is flagged for review queue via a `flag_for_review ToolEffect`.

Streaming with citation. As the Lecturer streams, citation markers like [P3] appear inline in the text. The frontend resolves these to hoverable/tappable references (spec'd in Frontend). No structured citation tool call is needed during streaming; the citations are text markers that the client parses.

Persistence. After a segment completes, the Orchestrator applies a `record_content_artifact ToolEffect` that writes the segment to `content_artifacts` with `kind = 'lecture_segment'`, `stance = <stance>`, `body = <streamed text>`, and one `content_citations` row per resolved [Pn] marker. This is what makes the segment reusable: the next learner on the same concept, at the same stance, reads the cached artifact rather than paying for regeneration. The Curator's `select_stance` decides whether to reuse or regenerate based on artifact age, review status, and the learner's specific context.

Comprehension checks. After every second segment, the Lecturer generates a check via a separate non-streaming call with `kind = 'generate_check'`. Output schema:

```
class ComprehensionCheck(BaseModel):
    question: str
    expected_key_points: list[str]      # 1-3 items
    hint: str
    model_answer: str                  # revealed only after learner
    attempts
```

The check is presented via the frontend; the learner's answer is graded by the Evaluator (§12) with `kind = 'grade_check'`.

Cost profile. ~10-25k input tokens per segment (mostly cached after the first segment on a concept), ~400-800 output tokens. Cache hit rate should exceed 90% within a session on a given concept.

11. The Tutor

The Tutor conducts Socratic dialogue. This is the heart of the product, per the product plan, and its behavior distinguishes Studium from every “chatbot with a system prompt” that has come before.

Kinds accepted.

Kind	Purpose	Streaming
------	---------	-----------

Kind	Purpose	Streaming
open_tutorial	Start a tutorial session on a concept	Yes
answer	Reply to a learner's turn in an ongoing tutorial	Yes
interruption_response	Reply to a lecture interruption; the "raise your hand" flow	Yes
office_hours_response	Reply during an office_hours session (looser, learner-driven)	Yes
primitive:<name>	Handle a specific interaction primitive	Yes
remediation	Take over when Lab has repeated failures on the same concept	Yes

System prompt structure.

[CACHED PREFIX – byte-stable per (concept_id, grounding_version)]

You are the Tutor for Studium. You conduct one-on-one Socratic dialogue with a serious adult learner. Your job is not to explain but to ask – to draw the learner into producing the argument themselves, and to notice where the argument breaks so you can push them past that specific point.

CONCEPT
{concept.title}

WHAT THE LEARNER SHOULD BE ABLE TO DO
{learning objectives from concept metadata}

CANONICAL SOURCES (reference internally; cite by [Pn] if you quote)
{retrieved passages, numbered}

THE DIAGNOSTIC SEQUENCE (from the product plan §5.4)
When the learner asks a question or answers one, run this internally:

1. CLASSIFY the question or answer:
 - Vocabulary: they do not know a word or notation.
 - Substance: they do not know an idea.
 - Scope: they are uncertain when the idea applies.
 - Justification: they want to know why a claim is true.
 - Connection: they want to see how this relates to something else.

Different classifications get different pedagogical moves.
2. CHECK CONTEXT: what did they recently struggle with (from the open journal entries in your context)? What does mastery say they are prepared for?
3. CHOOSE THE PEDAGOGICAL MOVE:
 - A direct answer, when the classification is Vocabulary or when a question is clearly outside the current concept's scope.

- A return question, when the learner is close and a small prompt will let them arrive at the answer.
 - A worked micro-example, when the learner is missing a concrete anchor.
 - A change of representation, when the learner is stuck in one framing and would benefit from a picture, a table, a code snippet, or a physical analogy.
 - A "let me show you where this fails", when the learner has a wrong generalization that a counterexample would sharpen.
4. RESPOND. Grounded in the sources when factual. In your own words when conceptual. Never fabricate citations.
5. CLOSE THE LOOP. A brief check: "does that land," "can you state it back to me," "shall we try one." The next turn either updates mastery evidence or opens a new sub-thread.

TONE

- Patient. Never condescending. Never sycophantic.
- Concise. Under 200 words unless the learner explicitly asks for depth.
- Use the learner's phrasing when reflecting back. Do not correct their informal terms unless the informality is the source of confusion.
- If the learner is wrong, say so directly, then work with them toward right. "That is not right – the trap here is that ... let me show you" is better than "Great question! One consideration is ..."

DO NOT

- Do not explain what the learner already understands. If mastery says they know it, assume they know it.
- Do not summarize your own answer at the end. The learner reads what you write; they do not need it repeated.
- Do not respond to a wrong answer with a right answer. Respond with the question that would have revealed the wrongness.
- Do not invent facts. If you need a fact you do not have, say so.

[PER-CALL SUFFIX]

Mode: {mode}

Current concept mastery: {p_known_decayed} (0=unknown, 1=known cold)

Open journal entries for this concept:

{list of open entries: summary, hypothesis, first_seen_at}

Recent session turns (last 6):

{formatted turns: actor, text, timestamp}

Task: {task-specific instruction, e.g. for 'answer':

"The learner just said: '{learner_utterance}'. Respond per the diagnostic sequence."}

Interruption handling. When the Orchestrator transitions from LECTURING to PAUSED_FOR_QUESTION, it invokes the Tutor with kind = 'interruption_response' and a payload including the just-delivered lecture text (truncated to the last 800 chars) plus the learner's question. The Tutor's answer runs the diagnostic sequence but is bounded to ~150 words unless the learner signals they want to escalate. After the answer, the Orchestrator asks the learner if their question is resolved. If yes, the state machine returns to LECTURING with a two-sentence recap; if the learner escalates ("this needs more time"), the state machine transitions to OFFICE_HOURS.

Primitive handling. The eight interaction primitives (§13) all dispatch to the Tutor. Each has its own prompt suffix that instructs the Tutor how to handle that specific primitive. The primitive dispatch table maps primitive:explain_differently → _prompt_explain_differently(), etc.

Cost profile. ~5-15k input tokens per turn (highly cache-friendly), ~200-600 output tokens. Tutor sessions run 5-20 turns typically; cost per session is bounded but not trivial.

12. The Evaluator

The Evaluator grades. It is deliberately isolated from any agent whose tone might inflate grades — this is the “grading integrity” principle inherited from the existing draft's assess.py.

Kinds accepted.

Kind	Purpose	Streaming
grade_check	Grade a comprehension check answer (2-attempt cycle)	No
grade_practice	Grade a lab practice-problem attempt	No
grade_assessment	Grade a summative assessment attempt (one call, whole rubric)	No
check_partial	Give a rapid partial-correctness signal for live Lab feedback	No

System prompt structure.

```
[CACHED PREFIX – byte-stable per (concept_id, rubric_snapshot_hash)]
```

```
You are the Evaluator for Studium. You grade a learner's answer against a rubric. You judge substance, not tone. You do not encourage; you assess.
```

```
RUBRIC CRITERIA
```

```
{full rubric for the concept, with key_points and weights}
```

```
GRADING RULES
```

- 2 points: answer covers the key points accurately, in the learner's own words. Minor terminology differences are fine if substance is right.
- 1 point: partially correct. Some key points present, or minor factual inaccuracies, or an answer that gestures at the idea without articulating

```

    it.
- 0 points: missing, wrong, restates the question, or "I don't know."

- Award credit only for content actually present in the answer.
- Do not reward confident tone, length, or vocabulary without substance.
- List concretely which key points were missing or wrong.
- Feedback: 1-2 sentences per criterion, specific enough to study from.
- Never reveal a key point in your feedback that the learner did not
  attempt. Point in the direction ("the second condition needs
  attention") not the answer ("the second condition is that  $x < y$ ").

STRICT MODE
{if concept.metadata.strict_grading:
  "Grading is strict. Partial credit requires meaningful engagement with the
  key point, not merely mentioning a related keyword."}

[PER-CALL SUFFIX]

Learner answer:
{answer text}

Task: Grade per the rubric.

```

Structured output (matches GradeReport in the existing draft's models.py):

```

class CriterionGrade(BaseModel):
    criterion_id: UUID
    score: Literal[0, 1, 2]
    feedback: str
    missing_points: list[str]

class GradeReport(BaseModel):
    criterion_grades: list[CriterionGrade]
    overall_feedback: str

```

Score computation happens in application code, not by asking the model. This is the D3 decision ratified in the data layer v1.1 process: weighted mean of score / 2 * weight, normalized by sum(weights), with ungraded criteria excluded rather than counted as zero. Formalized in `studium.assessment.compute_score(report, rubric)` and `compute_pass(score, threshold)`.

Practice checks vs. assessments. `grade_check` and `grade_practice` are one-criterion grades (the segment's check or the lab problem's model answer); Haiku is sufficient and the whole call runs in under 3 seconds. `grade_assessment` is a whole-rubric grade (5-8 criteria); Opus 4.8 is used and the call takes 10-15 seconds. This split is why the model routing is per-kind (§18), not just per-agent.

Attempt cycle. A grade_check returning partially_correct or incorrect triggers a second attempt (with the hint revealed); a second incorrect reveals the model answer. This is the existing draft’s pattern; the runtime preserves it.

Persistence. For grade_assessment: writes assessment_attempts and assessment_responses rows (data layer §6.8). For grade_check and grade_practice: emits record_mastery_evidence ToolEffects that write to mastery_events (data layer §6.5).

Failure modes. Evaluator produces a grade outside the 0-2 range (schema will reject); a criterion_grade missing a criterion_id; a grade that would move a mastery estimate by more than 0.4 in a single evidence step (unusual, worth flagging). All three cases flag the trace to content_review_queue.

13. The Confusion-Tracker

The Confusion-Tracker runs asynchronously. It never appears to the learner. Its job is to notice patterns of gap that a single turn does not reveal — a learner who has answered “close but wrong” three times on adjacent concepts, or whose recent turns show a specific misconception recurring.

Kinds accepted.

Kind	Purpose	Streaming
evaluate_turn	Given a completed turn, decide whether a journal entry is warranted	No
revise_hypothesis	Given new evidence on an existing journal entry, refine the hypothesis	No
resolve_check	Given a resolved-looking journal entry, verify the underlying gap is closed	No

System prompt structure.

```
[CACHED PREFIX – byte-stable per concept]

You are the Confusion-Tracker for Studium. You never talk to the learner.
You watch turns and decide whether a pattern of confusion warrants a
journal entry that the Tutor should address later.

CONCEPT
{concept.title}

WHAT MASTERY OF THIS CONCEPT LOOKS LIKE
{learning objectives}

COMMON MISCONCEPTIONS ON THIS CONCEPT (from source material and reviewer
notes)
{list, if any curated}
```

WHAT COUNTS AS A GAP WORTH LOGGING

- The learner said something factually wrong that a nearby correct response did not correct.
- The learner asked a question whose answer they should have from the material they have already seen.
- The learner is stuck on a step (2+ failed attempts) that others at their mastery level typically pass.
- The learner's phrasing suggests a specific misconception (e.g., conflating alpha-equivalence with beta-equivalence for lambda calculus).

DO NOT LOG

- Ordinary novice confusion that a next turn resolves.
- Questions asking for information not yet covered.
- Requests to skip or slow down.

[PER-CALL SUFFIX]

Turn under evaluation:

{turn: actor, kind, input, output}

Prior turns (last 8):

{recent turns}

Open journal entries for this learner+concept:

{open entries}

Current mastery: {p_known_decayed}

Task: Decide whether this turn warrants a new journal entry, a hypothesis revision on an existing entry, or no action.

Structured output.

```
class TrackerDecision(BaseModel):
    action: Literal['none', 'create_entry', 'revise_entry', 'flag_resolved']
    entry_id: Optional[UUID] # required for revise/flag_resolved
    summary: Optional[str] # required for create_entry
    hypothesis: Optional[str] # required for create/revise
    origin: Literal['tracker_inferred'] = 'tracker_inferred'
    reasoning: str # internal
```

When it runs. The Orchestrator invokes the Confusion-Tracker asynchronously after every turn where the actor is learner, unless the turn is trivially a next intent. The call fires and forgets — the Tracker's decision is applied via ToolEffect in a background task, not on the critical path of the current turn.

Model. Haiku. This runs on every learner turn, so cost sensitivity is high. If the Tracker’s precision needs to improve, the escape hatch is to promote select high-signal calls to Opus based on rules (e.g., recurring wrong-answer patterns), not to move the baseline model up.

Cost profile. ~3-8k input tokens per call, ~100-200 output tokens. On Haiku, roughly \$0.01-0.02 per turn. Over a 90-minute session with 40-60 turns, adds up to a real line item, but at 3× less than any Opus-based baseline.

14. The Reviewer

The Reviewer runs spaced-repetition review sessions using FSRS. Its job is to select due cards, generate retrieval prompts that require the learner to produce (not select), grade responses using a lightweight Evaluator invocation, and update card state.

Kinds accepted.

Kind	Purpose	Streaming
select_cards	Given a learner and target session length, pick cards to review	No
generate_prompt	Given a card, produce a retrieval prompt (not a flashcard)	No
grade_response	Given a card and learner response, produce an FSRS rating	No
retrieval_check	Given a prior session’s summary, generate the session-open check	No

Card selection. Deterministic, in application code (`studium.review.selection`). Reads `review_cards` where `due_at <= NOW()` and `NOT suspended`, orders by due date, caps at the target count. Load-bearing concepts get priority weight. The Reviewer LLM is not involved in selection — it is a database query.

Prompt generation. For each card, the Reviewer generates a fresh retrieval prompt calibrated to the card’s stability and difficulty. High-stability cards get harder prompts (application, transfer); low-stability cards get more direct prompts (definition, statement).

[CACHED PREFIX – per concept]

You are the Reviewer for Studium. You generate short retrieval prompts for a spaced-repetition system. These are not flashcards; they are prompts that require the learner to produce an answer in their own words.

CONCEPT
{concept.title}

CANONICAL SOURCES
{retrieved passages}

PROMPT PRINCIPLES

- Ask for production, not recognition. Never multiple-choice.
- Vary the ask: definition, worked micro-example, connection to a related

- concept, application, edge case.
- One prompt at a time. Under 30 words.
- Do not give the answer in the prompt. Do not include the model answer in your output to the learner; put it in the structured field.
- If the concept is well-mastered (stability > 30 days), prefer application and transfer prompts. If freshly learned (stability < 5 days), prefer direct recall.

[PER-CALL SUFFIX]

Card state: stability={s}, difficulty={d}, days_since_last_review={n}
 Prior prompt on this card (if any): {last_prompt_text}

Task: Generate one retrieval prompt for this card.

Structured output.

```
class RetrievalPrompt(BaseModel):
    prompt: str
    kind: Literal['definition', 'worked_example', 'connection',
'application', 'edge_case']
    model_answer: str
    expected_key_points: list[str]
```

Grading and FSRS update. The learner's response is graded by a `check_partial` call to the Evaluator (Haiku, single-criterion). The Evaluator's verdict maps to an FSRS rating:

Evaluator verdict	FSRS rating
correct and confident	easy (4)
correct with hesitation or partial coverage	good (3)
partially_correct	hard (2)
incorrect	again (1)

The FSRS update is deterministic (`studium.review.fsrs.update()`), writing to `review_cards` and `review_events`.

Retrieval check at session open. Distinct from ordinary review. The Reviewer produces 2-3 prompts drawn from the prior session's `session_summaries.key_points` and `concepts_touched`, calibrated to test whether yesterday's material has stuck. Results feed the Curator's decision about whether to proceed with new material or revisit.

15. The tutorial primitives

Eight primitives, per the product plan §5.4. Each has a UI affordance (button or voice command; frontend spec) and a defined dispatch through the runtime.

Primitive	Agent	Behavior
<code>explain_differently</code>	Tutor first,	Tutor asks what about the current explanation is

Primitive	Agent	Behavior
	then Lecturer	not landing; based on that response, dispatches to Lecturer with kind='re_explain' and a different stance
prove_it_to_me	Tutor	Attempts to elicit the derivation from the learner first (“what would you expect the argument to look like?”); if learner cannot start, walks it
where_does_this_fit	Tutor	Places the concept in the graph neighbourhood; renders the relationship to prerequisites and dependents as text (frontend may render a graph fragment)
vocabulary_check	Tutor	Distinguishes terminology confusion from substance confusion; if terminology, offers a paraphrase and glossary link; if substance, opens a Socratic sub-thread
show_worked_example	Lecturer	Dispatches to Lecturer with kind='worked_example'; produces a fully worked example, step-by-step
let_me_try_one	Curator, then Lab	Curator selects a practice problem at the current mastery level; state transitions to LAB
why_does_this_matter	Tutor	Explains downstream utility — what this concept lets the learner do that they could not do before
im_lost	Tutor + Curator	Tutor asks what the last thing that made sense was; Curator resets the session focus to the last high-mastery concept in the neighbourhood; a brief bridge is delivered

Primitive dispatch code (orchestration/primitives.py):

```

async def dispatch_primitive(
    name: str, session_context: SessionContext, orchestrator: Orchestrator
) -> AsyncIterator[StreamChunk]:
    """Route a primitive invocation to the correct agent(s)."""
    handler = PRIMITIVE_HANDLERS.get(name)
    if handler is None:
        raise ValueError(f"Unknown primitive: {name}")
    async for chunk in handler(session_context, orchestrator):
        yield chunk

PRIMITIVE_HANDLERS = {
    'explain_differently': _handle_explain_differently,
    'prove_it_to_me': _handle_prove_it_to_me,
    'where_does_this_fit': _handle_where_does_this_fit,
    'vocabulary_check': _handle_vocabulary_check,
    'show_worked_example': _handle_show_worked_example,
    'let_me_try_one': _handle_let_me_try_one,

```

```

    'why_does_this_matter': _handle_why_does_this_matter,
    'im_lost':                _handle_im_lost,
}

```

Each `_handle_*` is a small async generator that composes the underlying agent calls. `_handle_explain_differently` for example:

```

async def _handle_explain_differently(ctx, orch):
    # Two-step: Tutor diagnoses, then Lecturer regenerates in new stance
    diagnostic = await orch.tutor.handle(AgentInput(
        session_context=ctx,
        kind='primitive:explain_differently',
        payload={'phase': 'diagnose'}
    ))
    yield StreamChunk(kind='text', payload={'text': diagnostic.text})

    # The diagnostic's structured output includes chosen new stance
    new_stance = diagnostic.structured.chosen_stance

    async for chunk in orch.lecturer.handle_streaming(AgentInput(
        session_context=ctx,
        kind='re_explain',
        payload={'stance': new_stance, 'segment_index':
            ctx.session.last_segment}
    )):
        yield chunk

```

16. Session lifecycle

Three phases: opening, body, closing. The body is where the state machine runs (§7); the opening and closing are one-shot procedures that bracket it.

Opening (`session/lifecycle.py::open_session`):

1. Verify budget (`session/budget_gate.py::pre_flight_check(user_id, mode)`). If daily hard cap exceeded, refuse with structured error including reset time.
2. Create `learning_sessions` row (data layer §6.6) with the mode and target duration.
3. Load `SessionContext` via `session/memory.py::assemble_context()`.
4. If a prior `session_summaries` row exists for this `learner_subject_id` and the last session was within 7 days:
 - a. Curator generates a session opening with retrieval-check prompts.
 - b. Run retrieval check: 2-3 prompts, learner answers, Evaluator grades, results written to `retrieval_checks` (data layer §6.11).
 - c. Retrieval-check results feed mastery updates as `review_correct/review_incorrect` events.
5. Curator selects initial focus concept and mode.
6. State machine enters the mode's initial state.

Body. State machine runs per §7. Every turn:

1. Learner input arrives.
2. Budget gate re-checked (cached; only revalidated every 5 turns unless approaching cap).
3. Intent classified if free-form.
4. State transition applied.
5. Agent dispatched.
6. Streaming output yielded to client via SSE.
7. Interruption watched for on the client side; if received, propagate to the streaming loop.
8. On completion: ToolEffects applied, `session_turns` row written, Confusion-Tracker fired async.

Closing (`session/lifecycle.py::close_session`):

1. Update `learning_sessions.ended_at` and `end_reason`.
2. Curator generates a session summary via `kind= 'summarize_session'`: what was covered, what was mastered, what remains open, what to expect next time. Writes to `session_summaries` (data layer §6.11).
3. Rerun mastery decay on the touched concepts (write `mastery_events` with `kind='decay_refresh'` where applicable).
4. Update `learner_subjects.current_focus_concept_id` to the Curator's next-topic choice.
5. Roll cost forward: sum this session's cost deltas across the five cost columns, write/upsert `cost_ledger` row for today.
6. Release the Orchestrator's in-memory state for this session.

17. Prompt caching strategy

Prompt caching is the difference between a \$150/month per-learner cost and a \$50 one, at MVP scale. The existing draft's `cached_system()` gets this right at the single-agent level; the multi-agent runtime extends the pattern.

Cache key composition. A cached prefix is byte-stable across calls when its key components are unchanged. The key varies per agent:

Agent	Cache key components
Curator	(<code>subject_id</code> , <code>subject_version</code> , <code>learner_id</code>)
Lecturer	(<code>concept_id</code> , <code>stance</code> , <code>grounding_version</code>)
Tutor	(<code>concept_id</code> , <code>grounding_version</code>)
Evaluator	(<code>concept_id</code> , <code>rubric_snapshot_hash</code>)
Confusion-Tracker	<code>concept_id</code>
Reviewer	<code>concept_id</code>

Agent	Cache key components
Orchestrator (intent)	(byte-stable; no variables)

grounding_version is a hash of subject.updated_at + concept.updated_at + max(source_chunks.updated_at) for the relevant chunks, computed once per session context assembly.

TTL choice. Anthropic offers 5-minute and 1-hour cache TTLs at different prices. Studium’s pattern:

- **5-minute TTL** for the Tutor’s per-concept prefix within an active tutorial exchange. Turns land within minutes; longer TTL wastes cache-write cost.
- **1-hour TTL** for Lecturer, Curator, and Confusion-Tracker prefixes. These agents make calls across longer intervals within a session and the 6× longer TTL amortizes.
- No caching for the Orchestrator’s intent classifier — the prompt is small and caching wouldn’t recoup the write cost.

Cache hit tracking. The agent_traces.cache_read_tokens and the two cache_write_*_tokens columns from data layer v1.1 §6.12 make hit rate a first-class metric. A per-session dashboard shows the ratio of cache-read tokens to total input tokens; a healthy pattern is 60-80% cache-read after the first three turns of a session.

Prefix stability discipline. A prefix that fails to hit cache when it should is a bug worth fixing, not accepting. Failure modes to watch:

- A field that “should be” byte-stable includes a floating-point value with variable trailing digits — always format such values with fixed precision.
- The retrieved passages come back in different orders — sort them by chunk_id before inserting into the prefix.
- A JSON-serialized list has non-deterministic dict key ordering — use json.dumps(..., sort_keys=True).

llm/prompts.py::build_prefix(agent, context) centralizes this; the offline test tier (from data layer §14) verifies that build_prefix(same_context) returns identical bytes across multiple invocations.

18. Model routing

Routing lives in llm/client.py::route(agent, kind), returning a model identifier.

Agent	Kind	Model	Rationale
Orchestrator	classify_intent	Haiku 4.5	Cheap, high-throughput classification
Curator	any	Opus 4.8	Sequencing decisions are load-bearing
Lecturer	deliver_segment, re_explain, worked_example	Opus 4.8	Learner-visible teaching quality dominates

Agent	Kind	Model	Rationale
Lecturer	generate_check	Opus 4.8	Check quality affects mastery signal
Tutor	all	Opus 4.8	Socratic dialogue quality is the product
Evaluator	grade_assessment	Opus 4.8	Full-rubric grading warrants the frontier
Evaluator	grade_check, grade_practice, check_partial	Haiku 4.5	Single-criterion; Haiku sufficient
Confusion-Tracker	all	Haiku 4.5	Runs on every learner turn; cost-sensitive
Reviewer	generate_prompt, retrieval_check	Opus 4.8	Prompt quality determines review value
Reviewer	grade_response	Haiku 4.5	Same rationale as Evaluator grade_check

Overrides. Any call site can override the default model in `AgentInput.payload.model_override`. Used sparingly, always with a reason tag that flows to the trace. Two known cases where override applies: (a) reviewer-flagged sessions may re-run the Confusion-Tracker on Opus to get a higher-signal hypothesis, (b) demonstrably-hard concepts may permanently promote the Confusion-Tracker to Opus via a per-concept flag in `concepts.metadata`.

Provider abstraction. The `AnthropicClient` wrapper hides the SDK details behind a small interface (`stream`, `parse`, `count_tokens`). If a future model change wants a different provider for one agent, the substitution happens at this layer with no agent code change. Not built for MVP; noted for the moment it matters.

19. Cost accounting

Every LLM call writes an `agent_traces` row (data layer §6.6). The runtime enforces this by making the trace write part of the `AnthropicClient` wrapper, not a caller responsibility:

```
class AnthropicClient:
    async def stream(self, agent: str, model: str, **kwargs) -> AgentOutput:
        started = time.monotonic()
        response = await self._sdk.messages.stream(model=model, **kwargs)
        # ... consume stream, collect chunks ...
        latency_ms = int((time.monotonic() - started) * 1000)
        cost = compute_cost(model, response.usage)
        await traces.write(TraceRecord(
            agent=agent, model=model,
            prompt_messages=kwargs['messages'],
            system_prompt_hash=sha256(kwargs['system']),
            completion=full_text,
            tokens_in=response.usage.input_tokens,
```

```

        tokens_out=response.usage.output_tokens,
        cache_read_tokens=response.usage.cache_read_input_tokens,
        cache_write_5m_tokens=response.usage.cache_creation_5m_tokens,
        cache_write_1h_tokens=response.usage.cache_creation_1h_tokens,
        latency_ms=latency_ms,
        cost_usd=cost,
        langfuse_trace_id=current_langfuse_trace(),
    ))
    return AgentOutput(...)

```

Cost attribution to cost_ledger. The runtime does not update cost_ledger synchronously on every trace — that would create write contention on a small hot table. Instead, a daily job (studium.jobs.cost_rollup) reads yesterday’s agent_traces and the four other source tables (content_artifacts, ingestion_jobs, session_summaries, assessment_attempts.grading_cost_usd) and produces one cost_ledger row per (user_id, day, model). Same-day tracking uses live sums via studium.cost.today_spent() (data layer v1.1 §8).

Budget enforcement. session/budget_gate.py::pre_flight_check:

```

async def pre_flight_check(user_id: UUID, expected_max_usd: float) -> None:
    caps = await user_budget_caps.get(user_id)
    today = await cost.today_spent(user_id)
    month = await cost.month_spent(user_id)

    if today + expected_max_usd > caps.daily_hard_usd:
        raise BudgetExceededError(
            scope='daily', limit=caps.daily_hard_usd, spent=today,
            reset_at=start_of_tomorrow(user_tz)
        )
    if month + expected_max_usd > caps.monthly_hard_usd:
        raise BudgetExceededError(
            scope='monthly', limit=caps.monthly_hard_usd, spent=month,
            reset_at=start_of_next_month(user_tz)
        )

    if today + expected_max_usd > caps.daily_soft_usd:
        # Not raised; passed as a warning that the frontend renders
        session_context.warnings.append(BudgetWarning(
            scope='daily', limit=caps.daily_soft_usd, spent=today
        ))

```

expected_max_usd is a per-mode estimate: lecture≈\$0.25, tutorial≈\$0.50, lab≈\$0.30, review≈\$0.15, summative_assessment≈\$0.75. These are conservative overestimates so the gate errs toward blocking early rather than blocking mid-session.

20. Concurrency, streaming, and interruption

Streaming architecture.

```

FastAPI handler (async)
├── Orchestrator.handle_turn() → AsyncIterator[StreamChunk]
│   └── Lecturer.handle_streaming() → AsyncIterator[StreamChunk]
│       └── AnthropicClient.stream() → chunks from SDK
└── SSEmitter → yields `data: {...}\n\n` to the response body

```

Each layer is an `AsyncIterator`. Backpressure is handled naturally: if the client stops consuming, `await response.write()` blocks, which blocks the emitter, which blocks the agent's stream loop, which blocks the SDK. No explicit backpressure signal.

Interruption signal. The client sends an interrupt via a separate short POST to `/api/session/{id}/interrupt`, not via the streaming channel (SSE is one-directional). The Orchestrator holds a `asyncio.Event` per active session; setting the event triggers the streaming loop to enter its “finish current sentence” behaviour.

Sentence-boundary detection.

`orchestration/streaming.py::sentence_boundary_iter(chunks)` wraps the underlying token stream, buffers tokens, and emits full sentences. On interrupt, it stops requesting new tokens after the next completed sentence emission. Sentence boundary detection uses a rule-based algorithm (regex over `. ! ?` followed by whitespace and a capital letter, with an exception list for abbreviations); if the buffered text after 60 additional tokens still has no boundary detected, the Orchestrator falls back to a Haiku call (`orchestration/streaming.py::_llm_boundary_check`) to identify the boundary.

Cost of interruption. The tokens emitted after the interrupt signal but before the sentence boundary are wasted (paid for, not delivered to the learner). Typical waste: 20-80 tokens. Acceptable, and the user-perceived quality of finishing the sentence justifies the cost.

Concurrent sessions per user. Studium supports at most one active session per user. Starting a session while another is active either resumes the active one (if the mode matches) or asks the learner to close the existing session first.

Concurrent agent calls within a session. The Orchestrator serializes agent calls within a session — one turn's dispatch completes before the next begins. The exception is the Confusion-Tracker, which runs after a turn completes but does not gate the next turn.

21. Error handling and degradation

Failure modes and their designed responses.

Failure	Response
Model API timeout	Retry once with exponential backoff (500ms, then 2s). On second failure, degrade: text notice “the tutor is thinking longer than usual — please repeat or try again in a moment”

Failure	Response
Model API 5xx	Same as timeout
Rate limit (429)	Retry after Retry-After header. If second retry also 429, surface to the learner as “high demand right now; try again in a moment”
Content filter trip	Log the trace to content_review_queue with severity 3. Reword prompt with a “please rephrase” appendix and try once. On second trip, ask the learner to rephrase their input
Budget hard cap exceeded	Structured error, learner-visible: “You’ve reached today’s usage limit. Your progress is saved; resume tomorrow.” Include reset time in the learner’s timezone
Structured output parse failure	Log full response to trace with severity 3. Retry once with an explicit “your previous response did not parse; ensure the tool call schema is followed” appendix. On second failure, fall back to a hand-parsed extraction where possible; otherwise apologize and ask the learner to try again
Database write failure during ToolEffect application	Rollback the whole ToolEffect batch; log; the turn’s trace still writes. Learner sees a “your last answer was received but could not be recorded — please tap ‘confirm’ to retry” prompt
Client disconnects mid-stream	Cancel the underlying SDK call; save the partial output as a session_turns row with end_reason='client_disconnect'; do not retry (client will reconnect and either resume or restart)
Sentence-boundary detector runaway (60+ tokens after interrupt with no boundary)	LLM boundary check; if that also fails, cut at 80 tokens post-interrupt and mark the trace

Degradation copy is centralized in `studium/copy/degradation.py` so tone is consistent and localizable later.

Escalation to reviewer. Traces flagged for review land in `content_review_queue` (data layer §6.13) with severity:

- 1 (low): unusual pattern, worth a look
- 2 (medium): defect suspected in generated content
- 3 (high): grading anomaly, content filter trip, repeated structured output failure

The reviewer role (you, in MVP) sees the queue in the admin surface. Nothing about queue mechanics belongs in this spec beyond noting that agent runtime writes to it; the queue’s frontend is in the Frontend spec.

22. Observability

Every LLM call produces a Langfuse trace. Every FastAPI request produces an OpenTelemetry span. They share a `trace_id`.

Langfuse trace hierarchy.

```
session_id (top-level trace)
├── turn_index=0 (span)
│   ├── orchestrator.classify_intent (generation)
│   ├── curator.open_session (generation)
│   ├── reviewer.retrieval_check (generation)
│   └── evaluator.grade_check (generation)
├── turn_index=1 (span)
│   └── lecturer.deliver_segment (generation, streaming)
│       └── retrieve_passages (span, from Retrieval subsystem)
├── turn_index=2 (span)
│   └── confusion_tracker.evaluate_turn (generation, async)
...

```

Metadata attached to every generation. `agent`, `kind`, `session_id`, `user_id`, `focus_concept_id`, `cache_hit_rate`, `cost_usd`, `latency_ms`, `stop_reason`, `model_override_reason` (if applicable).

Per-agent dashboards (Langfuse; configured in Infrastructure spec):

- Lecturer: median segment latency, cache hit rate, cost per segment, review-queue flag rate
- Tutor: median turn latency, average turns per tutorial session, primitive invocation frequency
- Evaluator: grading distribution (0/1/2), pass rate, cost per assessment
- Confusion-Tracker: entries created per session, hypothesis revision frequency, resolved-vs-open ratio
- Curator: next-topic-lock rate (how often the Curator picks something that fails the unlock check)
- Orchestrator: intent classifier confidence distribution, interruption frequency, degradation event count

23. Testing strategy

Two tiers, matching the data layer's pattern.

Tier 1 — Offline (no LLM calls, no database).

- Prompt-prefix byte-stability: `build_prefix` returns identical bytes for identical inputs across multiple invocations. Verified with hash comparison.
- Structured output schema round-trip: every `AgentOutput.structured` model serializes to JSON and back losslessly.

- State machine transitions: enumerate all (state, event) pairs; assert the resulting state matches the transition table.
- Cost computation: `compute_cost(model, usage)` returns expected values for each model at each cache tier.
- Primitive dispatch: `dispatch_primitive('explain_differently', ...)` invokes the expected agent methods with the expected arguments (mocked agents).
- Budget enforcement: `pre_flight_check` raises `BudgetExceededError` at the correct thresholds.

Tier 2 — Online (requires live database and Anthropic API access).

- End-to-end session smoke test: open a session on a seeded lambda calculus subject, run 10 turns, close the session. Assertions: `session_turns` rows exist for each turn, `agent_traces` rows exist for each LLM call, `session_summaries` exists at close, mastery has moved on the focus concept.
- Interruption round-trip: start a lecture stream, send interrupt, verify a `PAUSED_FOR_QUESTION` state is reached, verify the Tutor produces a response, verify the state returns to `LECTURING` on resolution.
- Cache hit rate: after 10 turns on a single concept, verify `agent_traces.cache_read_tokens / agent_traces.tokens_in > 0.6` for the Lecturer and Tutor.
- Budget cap enforcement: set a very low cap, verify the second billable operation raises `BudgetExceededError` and the learner sees the correct degradation copy.
- Grading integrity: submit an intentionally wrong answer with confident tone, verify the Evaluator scores it 0.

Fixture agents. For Tier 1 tests, a `FakeAgent(kind → canned output)` class stands in for real agents. For Tier 2, the real agents run against a seeded database.

Regression tests on prompts. When a prompt is edited, a small regression suite runs a curated set of learner inputs through the affected agent and diffs the outputs against a snapshot. Differences require reviewer sign-off. This is the golden dataset flow that the Evaluation spec (subsystem 6) will formalize; this document only requires that the hooks exist.

24. Version history

v1.0 — 14 August 2026. Initial specification. Written against data layer spec v1.1. Locks the seven-agent structure, the session state machine, the eight tutorial primitives, the cached-prefix prompt architecture, the model routing table, and the cost accounting discipline. Explicit forward references to Retrieval (§6, Lecturer's `retrieve_passages`), Frontend (§20, SSE consumer; §21, degradation copy rendering), Content Ingestion (§10, artifact reuse policy), Evaluation (§23, regression on prompts), and Infrastructure (§22, Langfuse configuration).

Anticipated v1.1 candidates (not yet applied):

- If build reveals schema needs — e.g., a per-turn primitive column on `session_turns` beyond what data layer §6.6 already carries — record here and propose data layer v1.2.
 - If prompt-caching hit rates fall below 50% consistently, the prefix key composition in §17 needs revision.
 - If the interruption model’s “finish current sentence” produces visible artifacts (double sentences, missed detection), the sentence-boundary detector’s rule set needs iteration and the fallback threshold in §20 revisits.
-

25. Forward references and open questions

Items this spec deliberately punts to later specs or to build-time discovery.

Retrieval spec (subsystem 3). The interior of `retrieve_passages(concept_id, stance, k)`. This spec commits only that the tool exists, returns a list of ranked passages with `chunk_id` and `text`, and is called by Lecturer, Tutor, and Reviewer. Concrete details of chunking, embedding, hybrid ranking, and reranker parameters are downstream.

Frontend spec (subsystem 4). How SSE streams render. How the eight tutorial primitives are exposed as UI affordances. How the “raise your hand” interrupt is detected on the client (button, keyboard shortcut, voice trigger when voice ships). How degradation copy is rendered. How the concept graph, when it becomes a user-facing surface, integrates with the Curator’s next-topic decisions.

Content Ingestion spec (subsystem 5). How rubric criteria are authored. How concept metadata (learning objectives, common misconceptions, strict-grading flag) is populated. The per-artifact-kind `content_artifacts.metadata` schemas the runtime writes to.

Evaluation spec (subsystem 6). Golden datasets for prompt regression. Per-agent evaluation metrics. Reviewer tooling for the `content_review_queue` beyond what this document specifies at the agent-runtime boundary.

Infrastructure spec (subsystem 7). Langfuse project configuration and dashboards. Anthropic API key management. SSE proxying if a CDN sits in front. Auto-scaling triggers if MVP grows beyond a single Fly.io instance.

Open questions requiring build-time answers.

1. **Cache hit rate in practice.** The §17 target of 60-80% is a projection; actual rates depend on session shape and will be visible in Langfuse after week 1. If lower, the prefix composition may need tightening.
2. **Sentence-boundary detector accuracy.** The rule-based detector will almost certainly need tuning against real Lecturer output, especially on math-heavy segments. The fallback threshold (60 tokens) is a guess.
3. **Curator failure rate.** The unlock-check post-processing is a safety net; if it fires more than 5% of the time, the Curator’s system prompt needs sharpening on prerequisite reasoning.

4. **Confusion-Tracker precision.** How often does the Tracker create a journal entry that a reviewer subsequently marks as noise? Target under 20%; measurable from `content_review_queue` outcomes.
 5. **Interruption frequency.** How often do learners actually interrupt? If the answer is “rarely,” some of the state machine complexity in §7 is over-engineered; if “often,” the “finish current sentence” cost may need re-examination.
-

End of specification

This document defines the agent runtime for Studium in full. Seven agents, one state machine, eight interaction primitives, one streaming architecture, one cost accounting discipline. A senior engineer with the data layer v1.1 spec, this document, and access to Anthropic’s API can build a working runtime that produces the four MVP interaction modes end-to-end. The frontend that consumes the streaming interface is not required for the runtime to be complete; a curl-driven integration test suffices to demonstrate the runtime works.

Next in sequence: **Subsystem 3 — Retrieval and Knowledge Substrate**, which specifies the interior of `retrieve_passages`, chunking strategy, embedding model integration, hybrid search ranking, and citation resolution. It is written against data layer v1.1 and against this document’s contract for how retrieval is invoked.